Classification and Regression Trees III

Math 243

Michael Pearce

Outline

Last week, we...

- Introduced Regression trees
- Investigated cost complexity pruning to improve tree model accuracy
- Implemented trees in R

In today's class, we will...

- Discuss classification trees for classification problems.
- Build classification tree models in R

Section 1

Classification Trees

Classification Trees

Classification Trees: very similar to regression trees, except the leaves (or terminal nodes) predict the levels of a categorical response variable

Classification Trees

Classification Trees: very similar to regression trees, except the leaves (or terminal nodes) predict the levels of a categorical response variable

- Split up the data into groups based on predictors
- In each group, calculate the proportion of training observations in each level of categorical response variable
- **Classic Method:** Predict the most common level in a group

Quantifying Error

Recall: To grow a *regression tree*, we split the data into groups that minimize the sum of squared error (SSE):

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

Quantifying Error

Recall: To grow a *regression tree*, we split the data into groups that minimize the sum of squared error (SSE):

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

- SSE was our "metric" for determining if a particular split of the dataset was good

Quantifying Error

Recall: To grow a *regression tree*, we split the data into groups that minimize the sum of squared error (SSE):

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

- SSE was our "metric" for determining if a particular split of the dataset was good

To grow a *classification* tree, we make splits based on a metric *other* than SSE

Quantifying Error

Recall: To grow a *regression tree*, we split the data into groups that minimize the sum of squared error (SSE):

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

- SSE was our "metric" for determining if a particular split of the dataset was good

To grow a *classification* tree, we make splits based on a metric *other* than SSE

- Most natural choice is the Classification Error Rate (i.e., proportion of incorrect classifications):

$$\frac{1}{n}\left(\sum_{i \in S_1} I(y_i \neq \hat{y}_1) + \sum_{i \in S_2} I(y_i \neq \hat{y}_2) \right)$$

  where $\hat{y}_1$ and $\hat{y}_2$ are the most common classes in S_1 and S_2, respectively.

Quantifying Error

Recall: To grow a *regression tree*, we split the data into groups that minimize the sum of squared error (SSE):

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

- SSE was our "metric" for determining if a particular split of the dataset was good

To grow a *classification* tree, we make splits based on a metric *other* than SSE

- Most natural choice is the Classification Error Rate (i.e., proportion of incorrect classifications):

$$\frac{1}{n}\left(\sum_{i \in S_1} I(y_i \neq \hat{y}_1) + \sum_{i \in S_2} I(y_i \neq \hat{y}_2) \right)$$

  where $\hat{y}_1$ and $\hat{y}_2$ are the most common classes in S_1 and S_2, respectively.

But Classification Error Rate is not as "sensitive" as we would like...

(In)sensitivity of Classification Error Rate: Example

Consider a case with a binary Y

(In)sensitivity of Classification Error Rate: Example

Consider a case with a binary Y

- In the data, we have **300 failures (0's)** and **500 successes (1's)**
- Without any splits, we predict "1" for everyone, which gives a classification error rate of 300/800

(In)sensitivity of Classification Error Rate: Example

Consider a case with a binary Y

- In the data, we have **300 failures (0's)** and **500 successes (1's)**
- Without any splits, we predict "1" for everyone, which gives a classification error rate of 300/800

Consider splitting the data into two groups:

- S_1 has **0 failures** and **100 successes**
- S_2 has **300 failures** and **400 successes**

(In)sensitivity of Classification Error Rate: Example

Consider a case with a binary Y

- In the data, we have **300 failures (0's)** and **500 successes (1's)**
- Without any splits, we predict "1" for everyone, which gives a classification error rate of 300/800

Consider splitting the data into two groups:

- S_1 has **0 failures** and **100 successes**
- S_2 has **300 failures** and **400 successes**

- Still predicts "1" for everyone, and has the same classification error rate of (300/800)

(In)sensitivity of Classification Error Rate: Example

Consider a case with a binary Y

- In the data, we have **300 failures (0's)** and **500 successes (1's)**
- Without any splits, we predict "1" for everyone, which gives a classification error rate of 300/800

Consider splitting the data into two groups:

- S_1 has **0 failures** and **100 successes**
- S_2 has **300 failures** and **400 successes**

- Still predicts "1" for everyone, and has the same classification error rate of (300/800)

...but this still feels like an improvement!

(In)sensitivity of Classification Error Rate: Example

Consider a case with a binary Y

- In the data, we have **300 failures (0's)** and **500 successes (1's)**
- Without any splits, we predict "1" for everyone, which gives a classification error rate of 300/800

Consider splitting the data into two groups:

- S_1 has **0 failures** and **100 successes**
- S_2 has **300 failures** and **400 successes**

- Still predicts "1" for everyone, and has the same classification error rate of (300/800)

...but this still feels like an improvement!

- We're doing perfectly in S_1
- Maybe we can further split up S_2 to do even better (i.e., grow the tree)

We need a "metric" that captures this as an improvement

Two Common Decision Metrics

Gini Index (G): Within a split of the data S_m,

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) \quad \text{where} \quad \hat{p}_{mk} = \text{prop. obs. in region m in class k}$$

- Gini Index is small if $\hat{p}_{mk}$ are close to 0 or 1.
- A measure of node impurity – if small, that means split S_m is mostly made up of observations from a single class

Two Common Decision Metrics

Gini Index (G): Within a split of the data S_m,

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) \quad \text{where} \quad \hat{p}_{mk} = \text{prop. obs. in region m in class k}$$

- Gini Index is small if $\hat{p}_{mk}$ are close to 0 or 1.
- A measure of node impurity – if small, that means split S_m is mostly made up of observations from a single class

Information or Entropy (D):

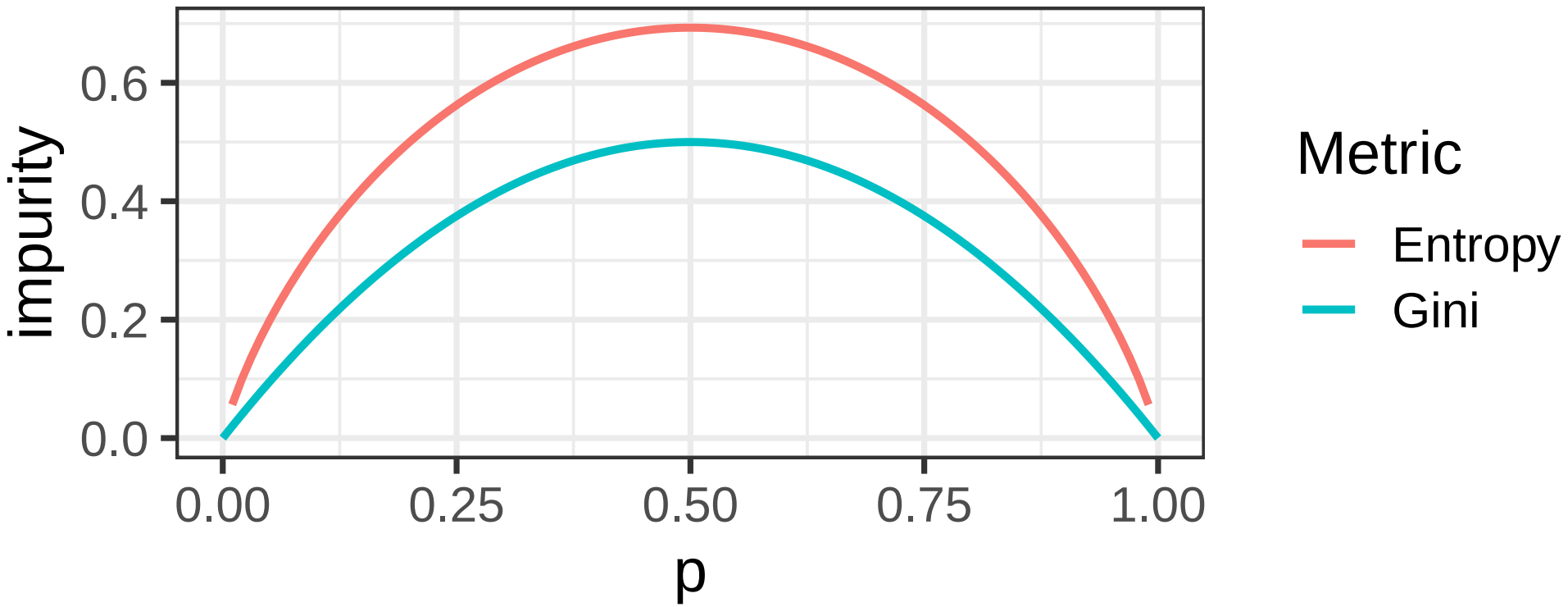
$$D = -\sum_{i=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk} \quad \text{where} \quad \hat{p}_{mk} = \text{prop. obs. in region m in class k}$$

- Entropy is small if $\hat{p}_{mk}$ are close to 0 or 1.

We want to minimize these "error" metrics!

Gini and Entropy

Gini Index (G) and Entropy (D) vs. $\hat{p}_{mk}$



Sensitivity in previous example

With a dataset with 300 failures (0's) and 500 successes (1's), and **no splits**

$$\text{Gini Index} = 2 * \frac{300}{800} * \frac{500}{800} = 0.468$$

$$\text{Entropy} = -\frac{300}{800}\log\frac{300}{800} - \frac{500}{800}\log\frac{500}{800} = 0.661$$

Sensitivity in previous example

With a dataset with 300 failures (0's) and 500 successes (1's), and **no splits**

$$\text{Gini Index} = 2 * \frac{300}{800} * \frac{500}{800} = 0.468$$

$$\text{Entropy} = -\frac{300}{800}\log\frac{300}{800} - \frac{500}{800}\log\frac{500}{800} = 0.661$$

Splitting up the data into $S_1 = (0 \text{ failures}, 100 \text{ successes})$ and $S_2 = (300 \text{ failures}, 400 \text{ successes})$:

$$\text{Gini Index} = \frac{100}{800}\underbrace{\left(2 * \frac{0}{100} * \frac{100}{100}\right)}_{\textit{Gini in } S_1} + \frac{700}{800}\underbrace{\left(2 * \frac{300}{700} * \frac{400}{700}\right)}_{\textit{Gini in } S_2} = 0.428$$

$$\text{Entropy} = \frac{100}{800}\underbrace{\left(-\frac{0}{100}\log\frac{0}{100} - \frac{100}{100}\log\frac{100}{100}\right)}_{\textit{Entropy in } S_1} + \frac{700}{800}\underbrace{\left(-\frac{300}{700}\log\frac{300}{700} - \frac{400}{700}\log\frac{400}{700}\right)}_{\textit{Entropy in } S_2} = 0.597$$

Sensitivity in previous example

With a dataset with 300 failures (0's) and 500 successes (1's), and **no splits**

$$\text{Gini Index} = 2 * \frac{300}{800} * \frac{500}{800} = 0.468$$

$$\text{Entropy} = -\frac{300}{800}\log\frac{300}{800} - \frac{500}{800}\log\frac{500}{800} = 0.661$$

Splitting up the data into $S_1 = (0 \text{ failures}, 100 \text{ successes})$ and $S_2 = (300 \text{ failures}, 400 \text{ successes})$:

$$\text{Gini Index} = \frac{100}{800}\underbrace{\left(2 * \frac{0}{100} * \frac{100}{100}\right)}_{\textit{Gini in } S_1} + \frac{700}{800}\underbrace{\left(2 * \frac{300}{700} * \frac{400}{700}\right)}_{\textit{Gini in } S_2} = 0.428$$

$$\text{Entropy} = \frac{100}{800}\underbrace{\left(-\frac{0}{100}\log\frac{0}{100} - \frac{100}{100}\log\frac{100}{100}\right)}_{\textit{Entropy in } S_1} + \frac{700}{800}\underbrace{\left(-\frac{300}{700}\log\frac{300}{700} - \frac{400}{700}\log\frac{400}{700}\right)}_{\textit{Entropy in } S_2} = 0.597$$

Both Gini and Entropy are better (lower) after this split!

Categorical Variables *as Predictors*

Trees can also use categorical predictors.

Categorical Variables *as Predictors*

Trees can also use categorical predictors.

Default Setting:

- Order categories by frequency (i.e., $X_k = \{A,\ B,\ C\}$ if A most common, C least common)
- Splits divide the collection of categories into 2 (i.e., $X_k \in \{A\}$ vs. $X_k \in \{B, C\}$)

Categorical Variables *as Predictors*

Trees can also use categorical predictors.

Default Setting:

- Order categories by frequency (i.e., $X_k = \{A,\ B,\ C\}$ if A most common, C least common)
- Splits divide the collection of categories into 2 (i.e., $X_k \in \{A\}$ vs. $X_k \in \{B, C\}$)

- Some interpretability lost
- With enough splits, we can still break up observations by each level of the categorical variable

Section 2

Classification Trees in R

Mushroom Hunting

Can I eat this?



Mushrooms

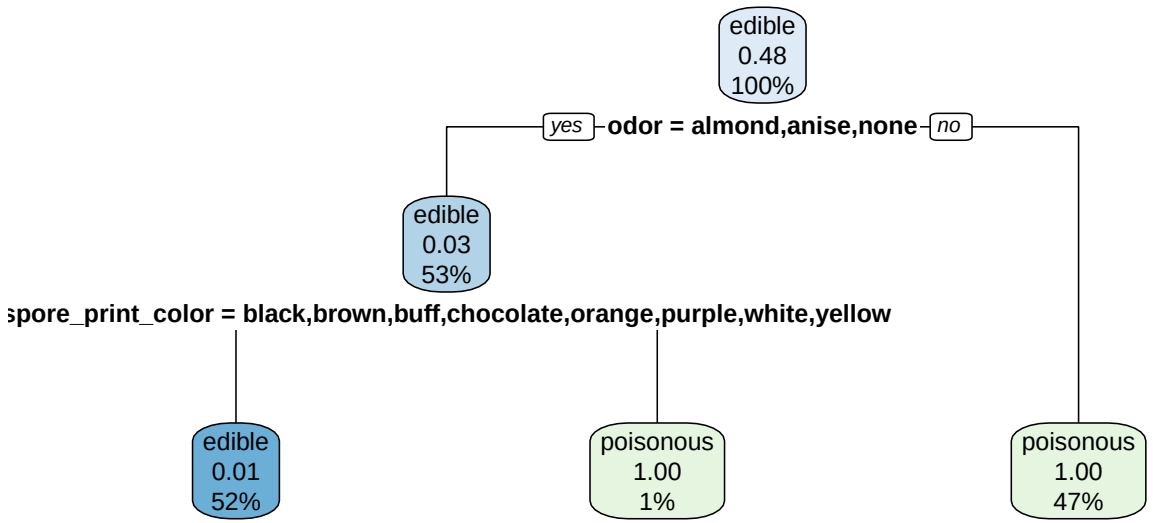
The `mushrooms` data set contains information on `edibility` (Y) and 22 other features (X) on 6,498 mushrooms.

```
## Rows: 6,498
## Columns: 23
## $ edibility                <fct> edible, edible, edible, edible, edible, edibl~
## $ cap_shape                <fct> convex, bell, convex, convex, bell, bell, bel~
## $ cap_surface              <fct> scaly, scaly, scaly, smooth, scaly, smooth, s~
## $ cap_color                <fct> yellow, white, gray, yellow, white, white, ye~
## $ bruises                  <fct> yes, yes, no, yes, yes, yes, yes, yes, yes, y~
## $ odor                     <fct> almond, anise, none, almond, almond, anise, a~
## $ gill_attachement         <fct> free, free, free, free, free, free, free, fre~
## $ gill_spacing             <fct> close, close, crowded, close, close, close, c~
## $ gill_size                <fct> broad, broad, broad, broad, broad, broad, bro~
## $ gill_color               <fct> black, brown, black, brown, gray, brown, gray~
## $ stalk_shape              <fct> enlarging, enlarging, tapering, enlarging, en~
## $ stalk_root               <fct> club, club, equal, club, club, club, club, cl~
## $ stalk_surface_above_ring <fct> smooth, smooth, smooth, smooth, smooth, smoot~
## $ stalk_surface_below_ring <fct> smooth, smooth, smooth, smooth, smooth, smoot~
## $ stalk_color_above_ring   <fct> purple, purple, purple, purple, purple, purpl~
## $ stalk_color_below_ring   <fct> purple, purple, purple, purple, purple, purpl~
## $ veil_type                <fct> partial, partial, partial, partial, partial, ~
## $ veil_color               <fct> white, white, white, white, white, white, whi~
## $ ring_number              <fct> one, one, one, one, one, one, one, one, one, ~
## $ ring_type                <fct> pendant, pendant, evanescent, pendant, pendan~
## $ spore_print_color        <fct> brown, brown, brown, black, black, brown, bla~
## $ population               <fct> numerous, numerous, abundant, numerous, numer~
## $ habitat                  <fct> grasses, meadows, grasses, grasses, meadows, ~
```

Implementing classfication trees in R with `rpart`

- Code to fit classification trees is same as regression trees!
- Default error metric: Gini Index

```
library(rpart)
library(rpart.plot)
mushroom_tree<-rpart(edibility ~ ., data = mushrooms)
rpart.plot(mushroom_tree)
```

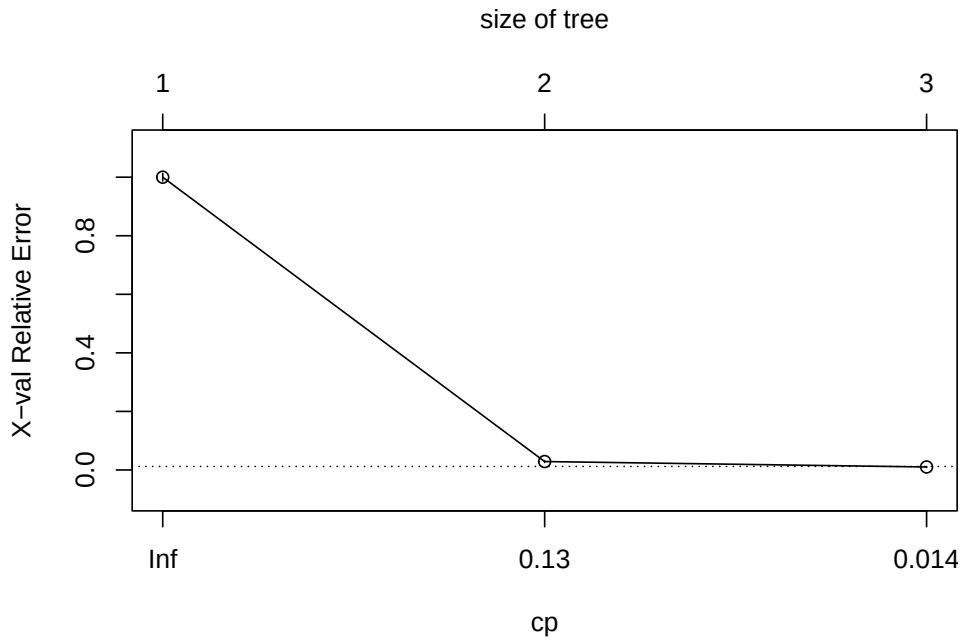


The default model has only 3 leaves and good class purity!!

CV Plot with `plotcp()`

We can plot the results of cross-validation using `plotcp`:

```
plotcp(mushroom_tree)
```



- Reminder: best to use *these* cp values when pruning the tree

Choosing the pruned tree with `prune()`

The tree with 3 leaves was better by a hair, but let's check out the tree with two leaves

- This tree had a $cp = 0.13$

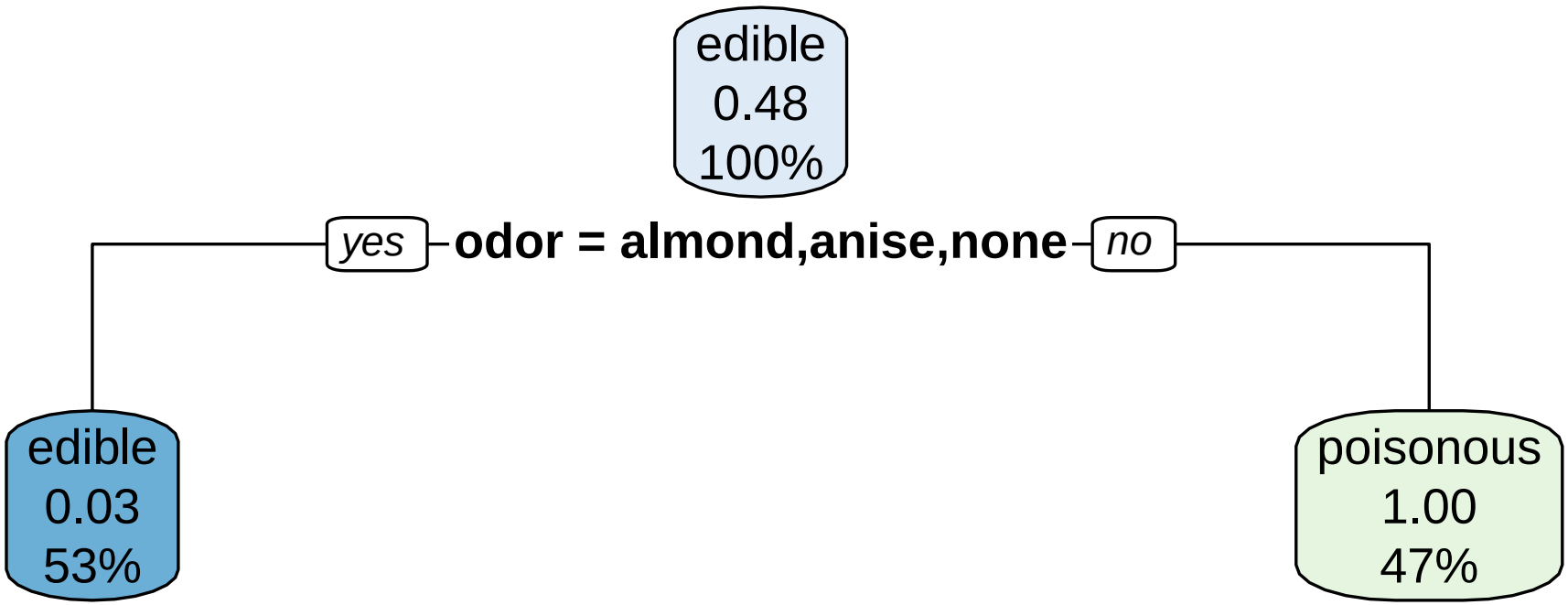
Choosing the pruned tree with `prune()`

The tree with 3 leaves was better by a hair, but let's check out the tree with two leaves

- This tree had a $cp = 0.13$

We can pull out the pruned subtree using the `prune` function with a given value of `cp`

```
mushroom_tree_short <- prune(mushroom_tree, cp = 0.13)
```



Test Accuracy

Suppose I obtained a new dataset of mushrooms – a test set! How good is my model?

Test Accuracy

Suppose I obtained a new dataset of mushrooms – a test set! How good is my model?

In the `predict()` function,

- `type="class"` gives predicted classes
- `type="prob"` gives probabilities of belonging to each class

Test Accuracy

Suppose I obtained a new dataset of mushrooms – a test set! How good is my model?

In the `predict()` function,

- `type="class"` gives predicted classes
- `type="prob"` gives probabilities of belonging to each class

```
mushroom_preds <- predict(mushroom_tree, mushrooms_test,
                          type = "class")
mushroom_probs <- predict(mushroom_tree, mushrooms_test,
                          type = "prob")[,"edible"]
mean(mushrooms_test$edibility == mushroom_preds)
```

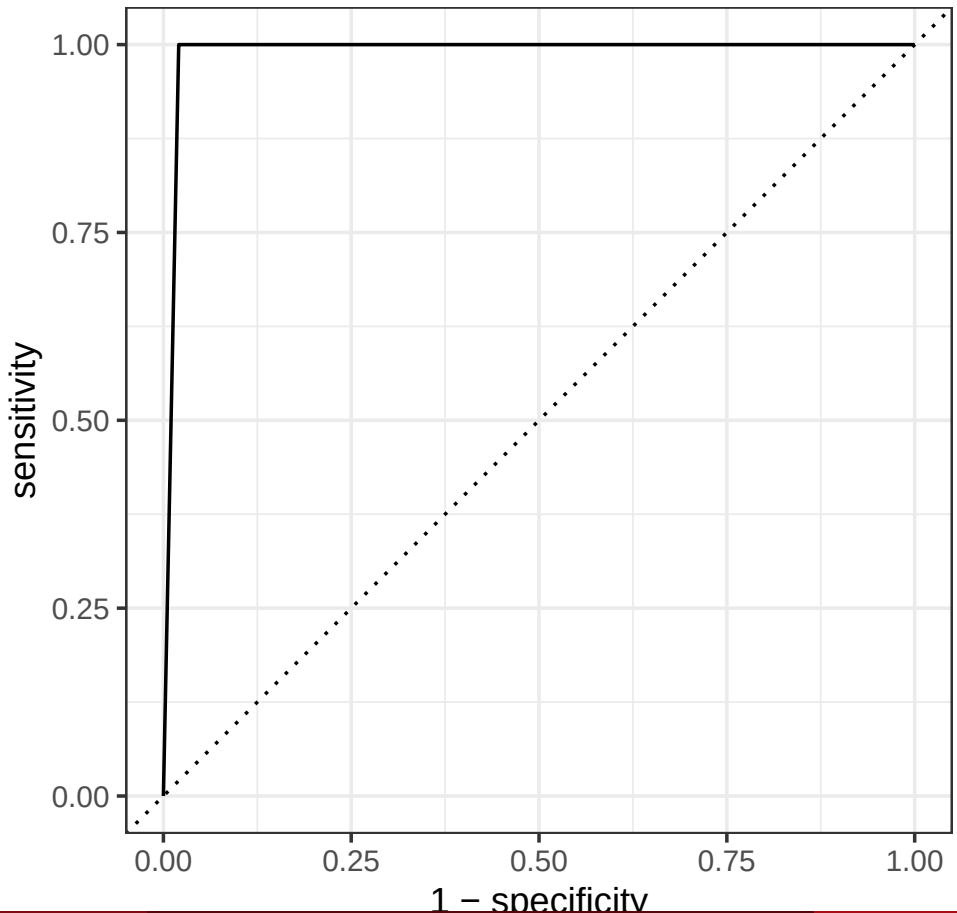
```
## [1] 0.9901599
```

Fantastic test accuracy!!

ROC Curve using yardstick package

Couldn't be any better!!

```
library(yardstick)
results <- data.frame(obs=mushrooms_test$edibility, preds=mushroom_preds, probs=mushroom_probs)
roc_curve(results, truth = obs, probs) %>% autoplot()
```



Confusion Matrix

```
table(Truth=mushrooms_test$edibility,
      Prediction=mushroom_preds)
```

```
##             Prediction
## Truth        edible poisonous
##   edible        842         0
##   poisonous      16       768
```

Confusion Matrix

```
table(Truth=mushrooms_test$edibility,
      Prediction=mushroom_preds)
```

```
##             Prediction
## Truth        edible poisonous
##   edible        842         0
##   poisonous      16       768
```

- We're dead if we get one of those 16 mushrooms!



Improvements

How can we reduce the rate of poison mushrooms identified as `edible`?

- *Option 1:* Everything is poisonous!
  - Downside: No tasty mushrooms :(

Improvements

How can we reduce the rate of poison mushrooms identified as `edible`?

- *Option 1:* Everything is poisonous!
  - Downside: No tasty mushrooms :(
- *Option 2:* Change classification threshhold
  - i.e. classify as edible only if $P(\text{edible}) > 99.9\%$

Improvements

How can we reduce the rate of poison mushrooms identified as `edible`?

- *Option 1:* Everything is poisonous!
  - Downside: No tasty mushrooms :(
- *Option 2:* Change classification threshhold
  - i.e. classify as edible only if $P(\text{edible}) > 99.9\%$
- *Option 3:* Incorporate "relative penalty" in Gini index.

$$G = \sum_i \sum_j L(i,j) p_i p_j$$

- $L(i,j)$ is the "Loss" incurred when predicting level j when the truth is level i.
- Set $L(i,i) = 0$: No penalty for being right!
- When "poisonous", predicting "edible"is *really* bad, so set $L(\textit{Poisonous}, \textit{Edible}) >>> L(\textit{Edible}, \textit{Poisonous})$.

Additional Parameters

To incorporate loss, create a penalty matrix and add to the `parms` argument in `rpart`:

```
penalty_matrix <- matrix(c(0,1,20,0), byrow = T, nrow = 2)
penalty_matrix
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]   20    0
```

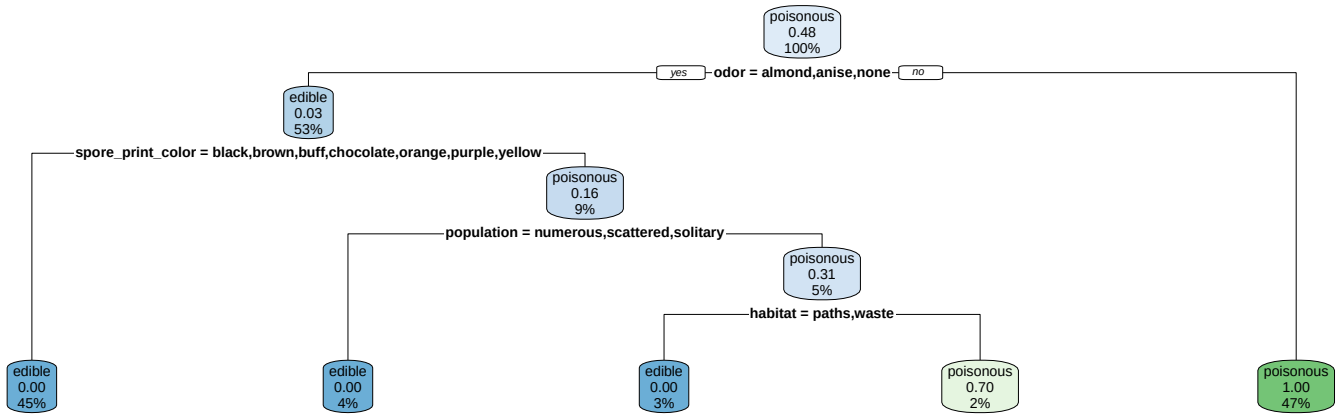
Additional Parameters

To incorporate loss, create a penalty matrix and add to the `parms` argument in `rpart`:

```
penalty_matrix <- matrix(c(0,1,20,0), byrow = T, nrow = 2)
penalty_matrix
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]   20    0
```

```
mushroom_no_poison <- rpart(edibility ~., data = mushrooms,
                            parms = list(loss = penalty_matrix))
rpart.plot(mushroom_no_poison)
```



New Results

```
mushroom_preds2 <- predict(mushroom_no_poison, mushrooms_test, type = "clas

mean(mushroom_preds != mushrooms_test$edibility) # original tree
```

```
## [1] 0.009840098
```

```
mean(mushroom_preds2 != mushrooms_test$edibility) # loss-based tree
```

```
## [1] 0.005535055
```

New Results

```
mushroom_preds2 <- predict(mushroom_no_poison, mushrooms_test, type = "clas

mean(mushroom_preds != mushrooms_test$edibility) # original tree
```

```
## [1] 0.009840098
```

```
mean(mushroom_preds2 != mushrooms_test$edibility) # loss-based tree
```

```
## [1] 0.005535055
```

No fatal errors anymore!

```
##             Prediction
## Truth        edible poisonous
##   edible        833         9
##   poisonous       0       784
```

New Results

```
mushroom_preds2 <- predict(mushroom_no_poison, mushrooms_test, type = "clas

mean(mushroom_preds != mushrooms_test$edibility) # original tree
```

```
## [1] 0.009840098
```

```
mean(mushroom_preds2 != mushrooms_test$edibility) # loss-based tree
```

```
## [1] 0.005535055
```

No fatal errors anymore!

```
##             Prediction
## Truth        edible poisonous
##   edible        833         9
##   poisonous       0       784
```

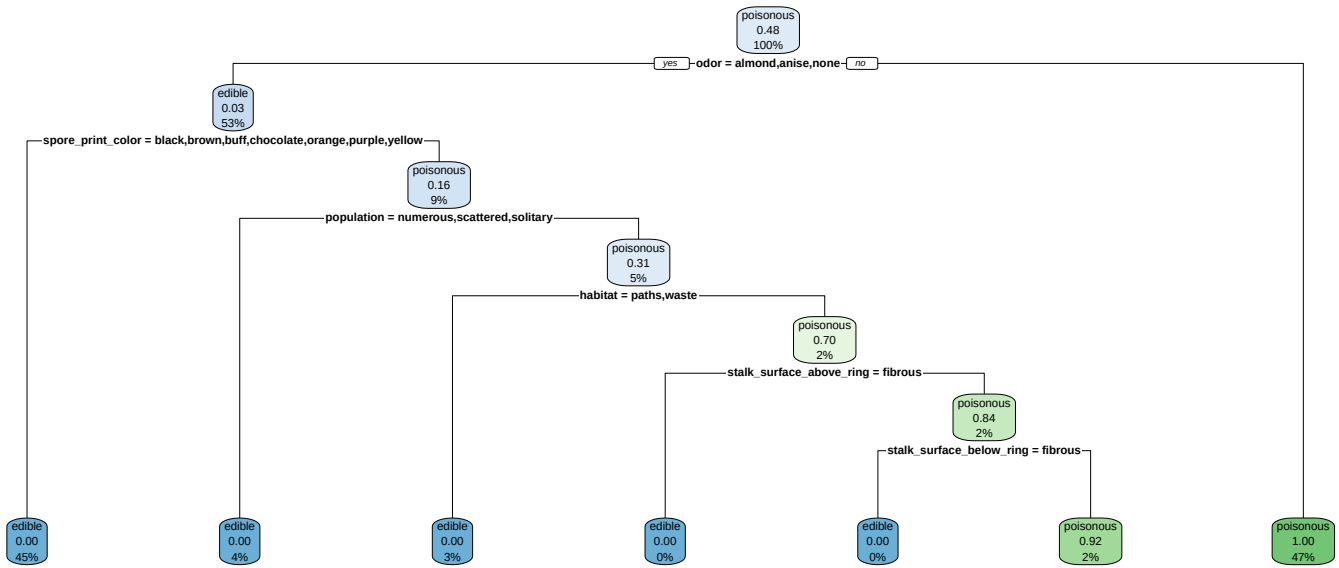
Could we somehow still improve the sensitivity?!

Build a Deeper Tree!

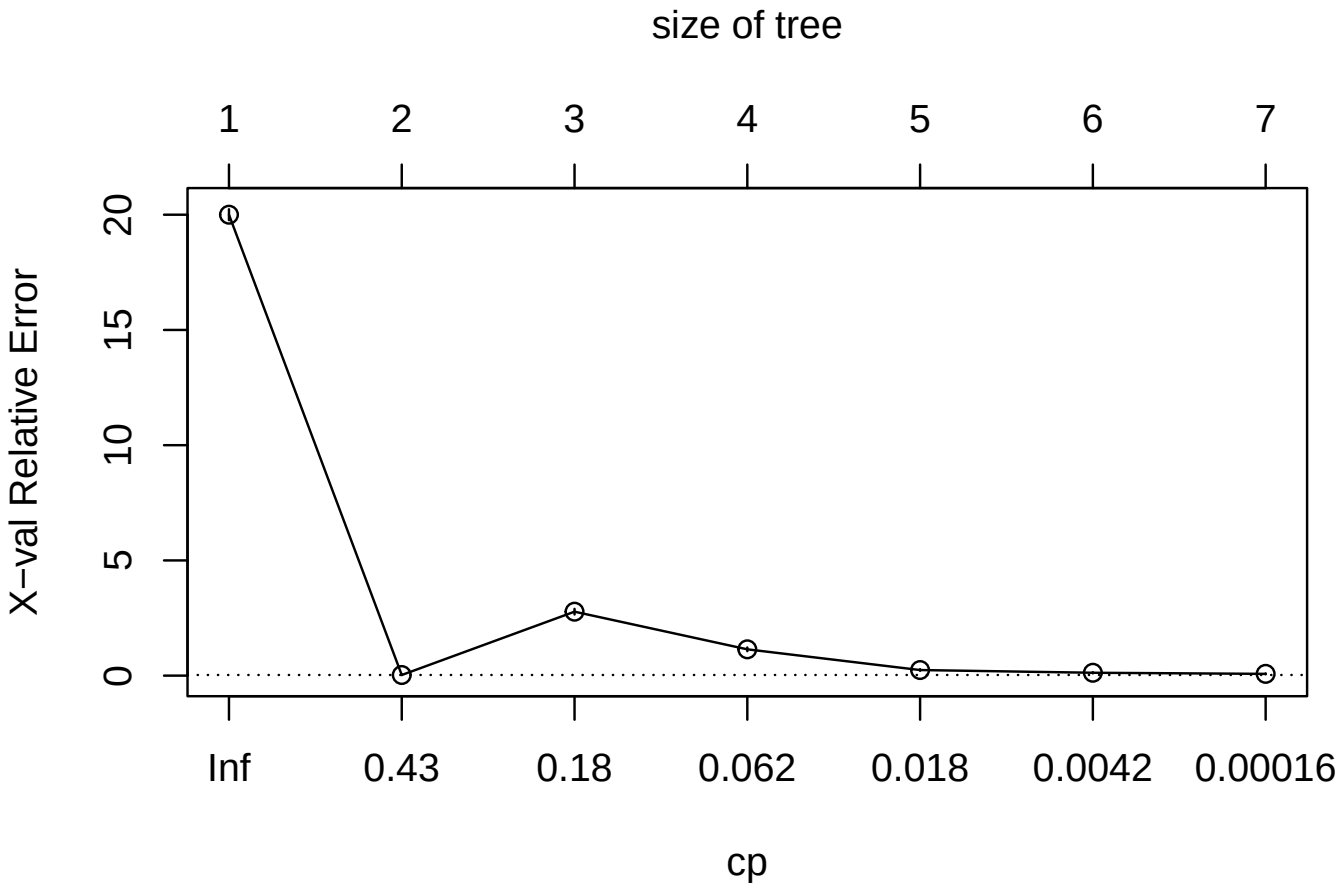
Control tree depth by setting the minimum cp parameter in `rpart.control`

- Any split that does not decrease overall lack of fit by a factor of cp is not attempted.
- Setting low values of cp lead to deeper trees

```
mushroom_deep <- rpart(edibility ~., data = mushrooms,
                       parms = list(loss = penalty_matrix),
                       control = rpart.control(cp = .00001))
rpart.plot(mushroom_deep)
```



Pruning



- To reduce chance of overfitting, let's prune to a tree with 6 leaves.

```
mushroom_prune <- prune(mushroom_deep, cp = 0.0042)
```

Final Results

```
mushroom_preds3 <- predict(mushroom_deep, mushrooms_test, type = "class")
mushroom_preds4 <- predict(mushroom_prune, mushrooms_test, type = "class")

accuracy_original<- mean(mushrooms_test$edibility == mushroom_preds)
accuracy_loss    <- mean(mushrooms_test$edibility == mushroom_preds2)
accuracy_deep    <- mean(mushrooms_test$edibility == mushroom_preds3)
accuracy_pruned  <- mean(mushrooms_test$edibility == mushroom_preds4)

round(c(accuracy_original,accuracy_loss,accuracy_deep,accuracy_pruned),3)
```

```
## [1] 0.990 0.994 0.998 0.996
```

Final Results

```
mushroom_preds3 <- predict(mushroom_deep, mushrooms_test, type = "class")
mushroom_preds4 <- predict(mushroom_prune, mushrooms_test, type = "class")

accuracy_original<- mean(mushrooms_test$edibility == mushroom_preds)
accuracy_loss    <- mean(mushrooms_test$edibility == mushroom_preds2)
accuracy_deep    <- mean(mushrooms_test$edibility == mushroom_preds3)
accuracy_pruned  <- mean(mushrooms_test$edibility == mushroom_preds4)

round(c(accuracy_original,accuracy_loss,accuracy_deep,accuracy_pruned),3)
```

```
## [1] 0.990 0.994 0.998 0.996
```

```
##            Prediction
## Truth       edible poisonous
##   edible       838         4
##   poisonous      0       784
```

Section 3

Recap

Recap

Today, we...

- Introduced Classification Trees
- Considered pruning of classification trees
- Implemented methods in R

Next time, we will...

- Discuss other ways of improving tree models using "ensemble" methods